C++

COPY CONSTRUCTOR, MOVE CONSTRUCTOR AND OVERLOADING ASSIGNMENT OPERATOR

BY: JANHAVI DEO

Copy constructor

- When an object is created using another object, copy constructor is called implicitly.
- Compiler provides default copy constructor in case the programmer is not providing.
- Default constructor does bit by bit copy of one object to another.
- It is called when we are passing object of any class as call by value parameter
- It is also called when any function is returning object
- We can create copy of object at the time of declaration. Syntax is

  date obj2(obj1)

  Here date is the name of class, obj2 is the new object to be created and it will be copy of obj1

```
class myArray{
  int size;
  int *ptr;

  public:

     myArray( ) {
        size = 5;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
           ptr[i] = 0;
     }

    ~myArray( ) {
        delete []ptr;
     }
};
```

Scenario 1:

```
int main( ) {
   myArray obj1;
   return 0;
}
```

Scenario 2:

```
int main( ) {
   myArray obj1;
   myArray obj2;
   return 0;
}
```

Scenario 3:

```
int main( ) {
   myArray obj1;
   myArray obj2(obj1);
   return 0;
}
```

```
class myArray{
  int size;
  int *ptr;

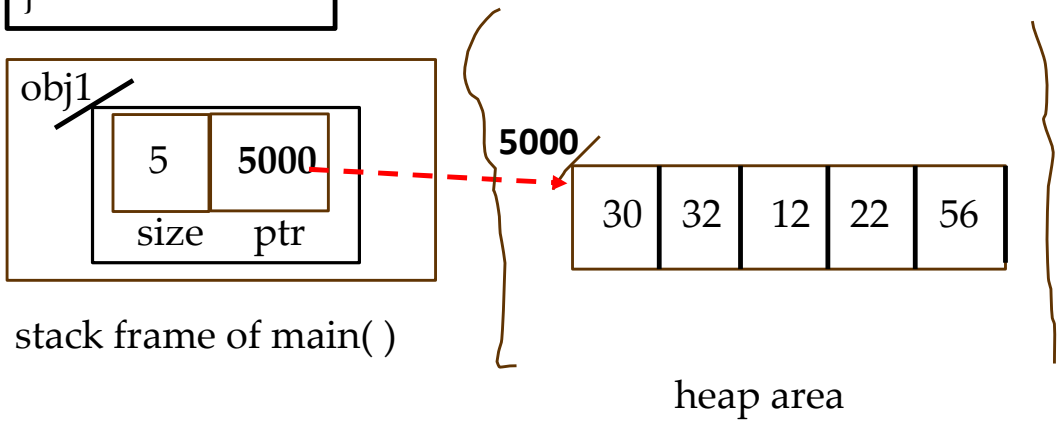
  public:

     myArray( ) {
        size = 5;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
           ptr[i] = 0;
     }

    ~myArray( ) {
        delete []ptr;
     }
};
```

Scenario 1:

```
int main( ) {
    myArray obj1;
    return 0;
}
```



```
class myArray{
  int size;
  int *ptr;

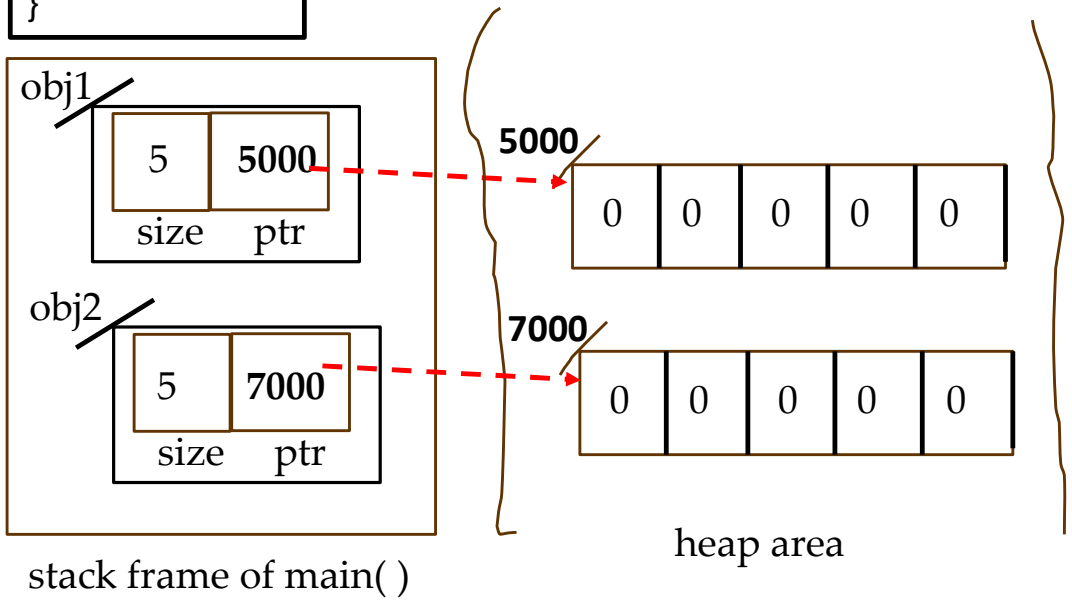
  public:

     myArray( ) {
        size = 5;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
           ptr[i] = 0;
     }

    ~myArray( ) {
        delete []ptr;
     }
};
```

Scenario 2:

```
int main( ) {
   myArray obj1;
   myArray obj2;
   return 0;
}
```



heap area

```
class myArray{
  int size;
  int *ptr;

  public:

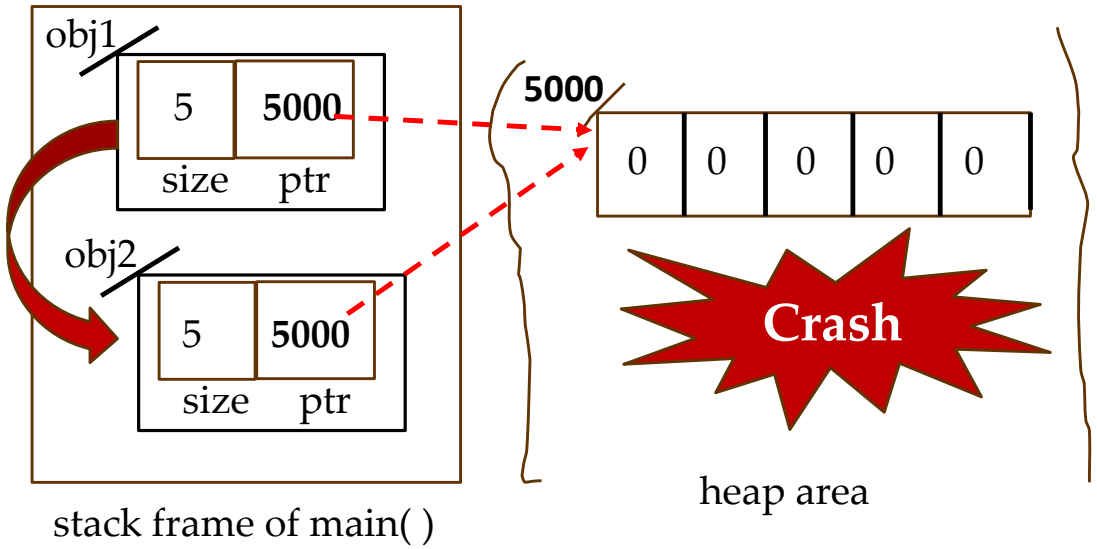
     myArray( ) {
        size = 5;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
           ptr[i] = 0;
     }

    ~myArray( ) {
        delete []ptr;
    }
};
```

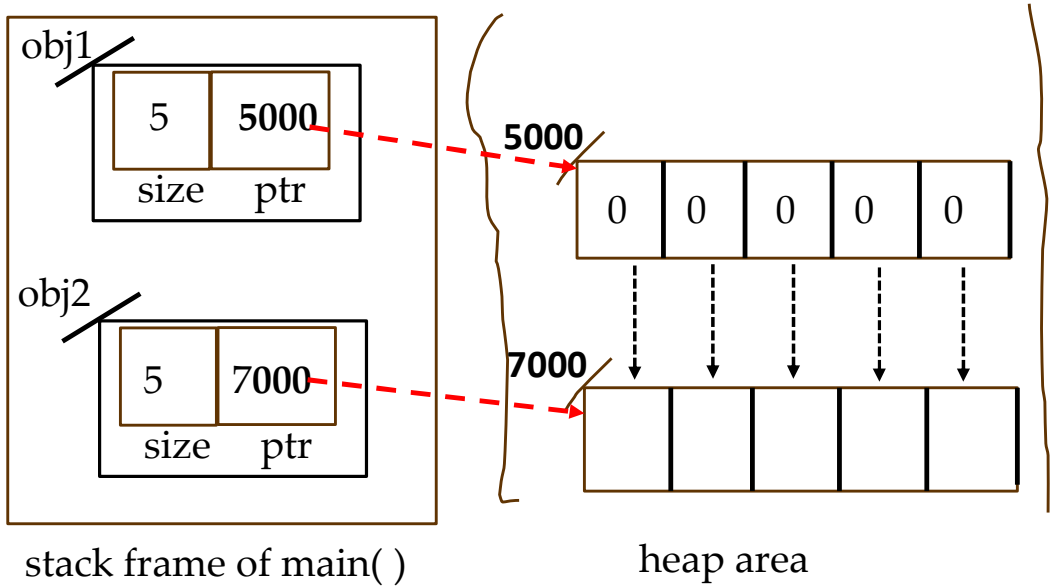
Shallow copy
Bit by bit copy

Scenario 3:

```
int main( ) {
   myArray obj1;
   myArray obj2(obj1);
   return 0;
}
```



Deep copy by copy constructor




```
myArray(myArray &m ) {
        size = m.size;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
            ptr[i] = m.ptr[i];
    }
```

- *Copy constructor has to copy data on stack from source object to target object*
- *Copy constructor has to allocate new memory for pointer variables in target object, then copy contents on heap area from source object to target object.*

Assignment operator

- When an object is assigned value of another object, assignment operator functions is called implicitly.
- Compiler provides default assignment operator implementation (=) in case the programmer is not providing.
- Default assignment operator function does bit by bit copy of one object to another.
- obj2 = obj1, when obj1 and obj2 are already declared objects of any class.

Consider scenario

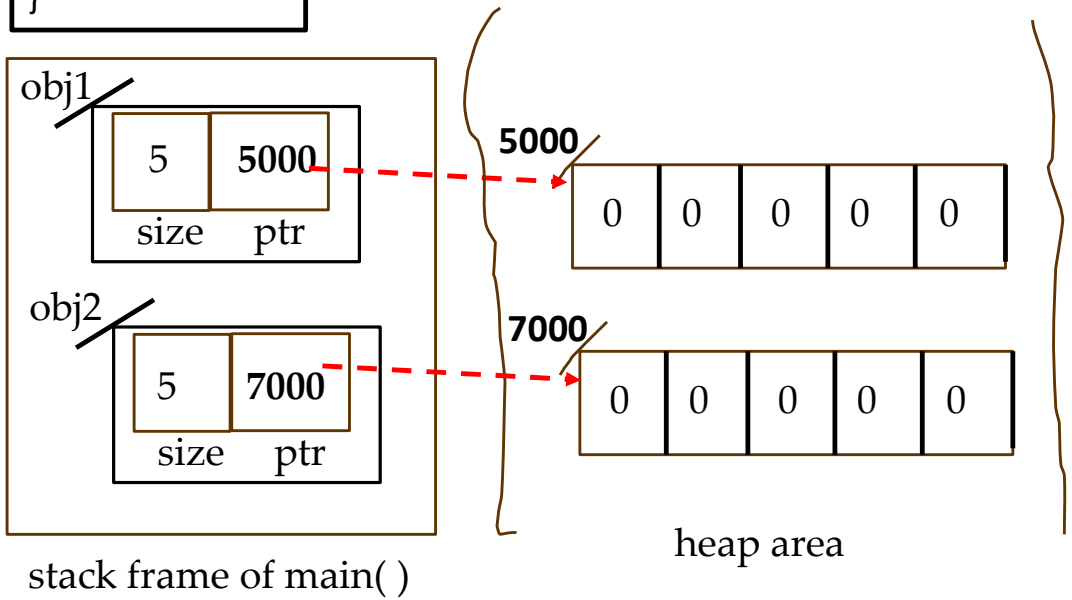
```
class myArray{
  int size;
  int *ptr;

  public:

     myArray( ) {
        size = 5;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
           ptr[i] = 0;
     }

    ~myArray( ) {
        delete []ptr;
    }
};
```

```
int main( ) {
    myArray obj1;
    myArray obj2;
    return 0;
}
```

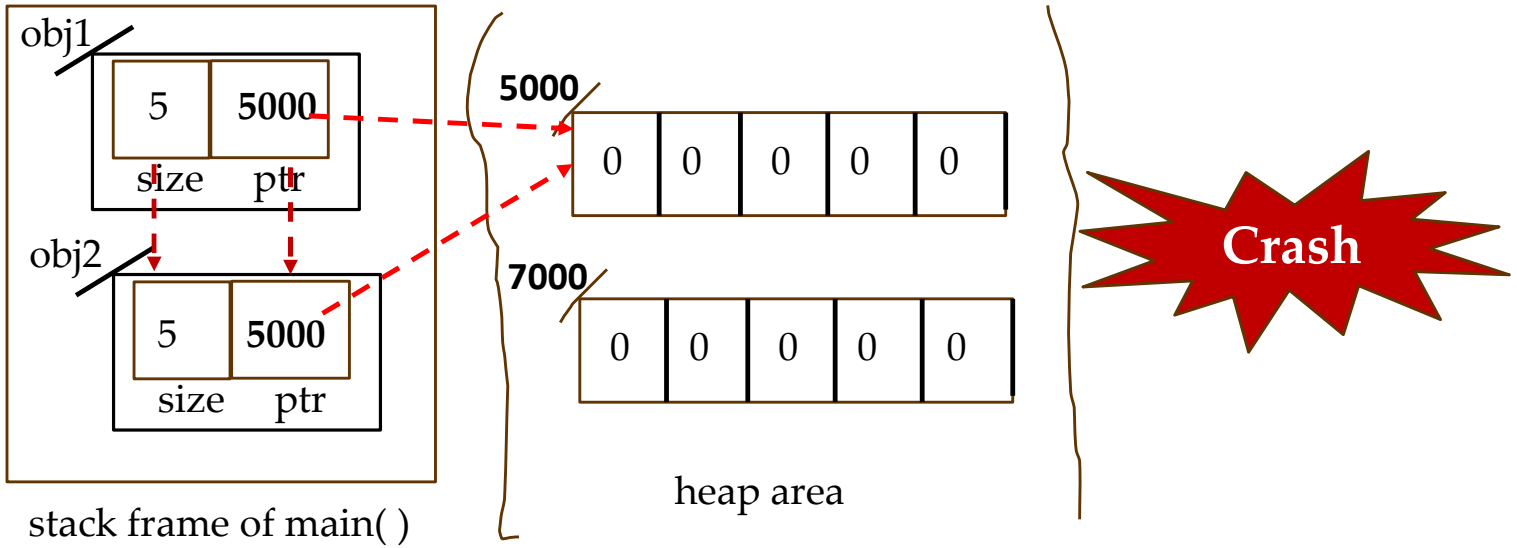


heap area

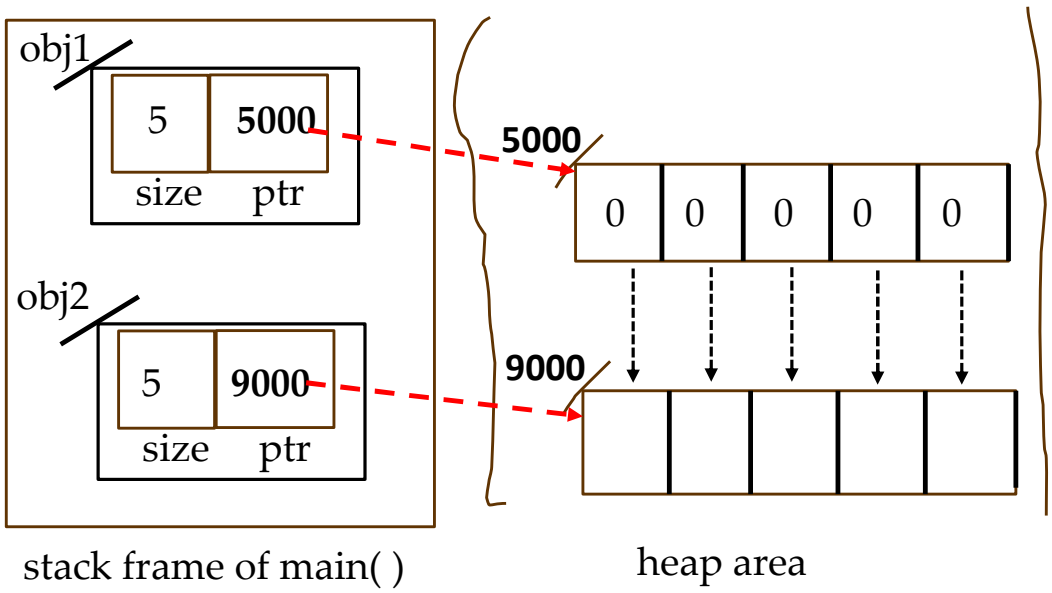
```
int main( ) {
   myArray obj1;
   myArray obj2;

   obj2 = obj1; //call to assignment operator
   return 0;
}
```

Default behaviour of assignment operator



Assignment Operator




```
myArray operator = (myArray &m ) {
        delete [ ] ptr;
        size = m.size;
        ptr = new int[size];
        for(int i=0 ;i<size ; i++)
            ptr[i] = m.ptr[i];
    }
```

- *Assignment operator has to release memory in hold for ptr of target object.*
- *Assignment operator has to allocate new memory for ptr of target object.*
- *Assignment operator has to copy values of source object to target object.*

Assignment operator

- Some compiler make it compulsory to define signature of assignment operator as

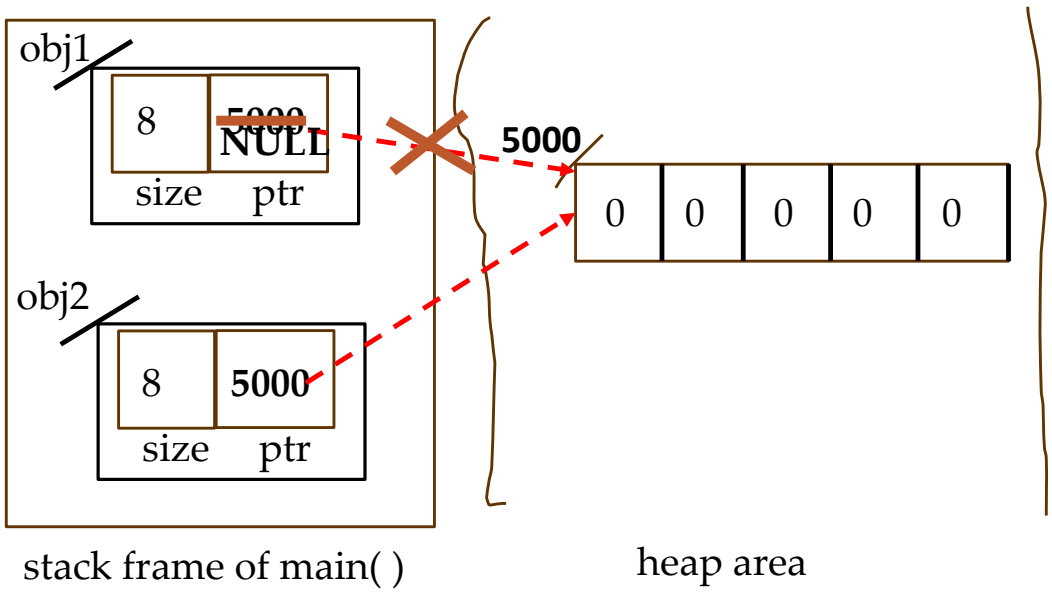
  const &myArray operator = (const myArray &m) { ……. }

  This means the function returns reference to constant and takes constant reference as parameter.

Move constructor

- A move constructor in C++ is a special constructor introduced in C++11.
- It transfers ownership of resources from one object to another instead of creating new and copying the values.
- It makes programs faster and more memory-efficient.
- It is typically invoked when working with temporary objects (rvalues) or when using std::move().

Move constructor




```
myArray (myArray &&m ) {
        size = m.size;
        ptr = m.ptr;
        m.ptr = nullptr;
    }
```

```
int main( ) {
   myArray obj1;
   myArray obj2 = move(obj1);
//myArray obj2 = std::move(obj1);
   return 0;
}
```

Move constructor

WHY TO USE :

- Transfers ownership of resources (like dynamic memory, file handles, sockets).
- Avoids expensive deep copies.
- Leaves the source object in a valid but "empty" state (commonly by setting pointers to nullptr).

When it will get Invoked:

- Returning an object by value.
- Initializing an object from a temporary.